Experiment: Classification with K-Nearest Neighbors Algorithm

Machine Learning and Applications

1. OBJECTIVE

To implement the K-Nearest Neighbors (K-NN) algorithm on a real-world morphological dataset and empirically deduce how the hyperparameter K influences the model's behavior. Students will process agricultural data, construct 2D decision boundaries across multiple values of K, and analyze the resulting learning curves to independently identify regions of overfitting and underfitting.

2. RELATED THEORY

2.1 The K-Nearest Neighbors Algorithm

K-Nearest Neighbors (K-NN) is a non-parametric, instance-based learning algorithm. Unlike parametric models (such as Linear Regression) that learn fixed mathematical equations, K-NN essentially "memorizes" the training dataset. To classify a new, unseen data point, the algorithm calculates the distance between that point and every point in the training set. It then identifies the K closest neighbors and assigns the class label determined by a majority vote among them.

Because K-NN relies on spatial geometry, distance is most commonly measured using the Euclidean distance formula:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

where p and q are two points in an n-dimensional feature space. Because this formula calculates absolute spatial differences, all features must be standardized prior to training to prevent variables with inherently larger scales from mathematically dominating the distance calculation.

2.2 Decision Boundaries

A decision boundary is a geometric hypersurface that partitions the feature space into distinct regions, with each region corresponding to a specific class label. In K-NN, these boundaries are implicitly defined by the training data and the chosen value of K. By evaluating the model across a dense grid of coordinates, we can visually map these regions and observe how the algorithm divides the spatial territory.

2.3 The Bias-Variance Tradeoff and Parameter K

The predictive power and generalization capability of a K-NN model are entirely governed by the hyperparameter K, which dictates the model's flexibility:

- **High Flexibility (Small K):** When K is very small (e.g., $K = 1$), the model predicts strictly based on the single most immediate neighbor. This results in highly complex, jagged decision boundaries that wrap around individual data points, capturing local anomalies and noise in the training data. This state is characterized by **high variance** and **low bias** (Overfitting).
- **Low Flexibility (Large K):** When K is large, the model's predictions are smoothed out over a broad neighborhood. This averages out the noise but can also cause the model to ignore valid, underlying local patterns. The decision boundaries become rigid and oversimplified. This state is characterized by **low variance** and **high bias** (Underfitting).

The fundamental goal of hyperparameter tuning is to perform a complexity sweep, plotting learning curves to identify the optimal K that minimizes validation error, thereby striking the ideal balance between bias and variance.

3. IMPLEMENTATION GUIDE

3.1 Data Acquisition and The Spatial Setup

This experiment utilizes the real-world Dry Bean Dataset sourced from Kaggle, which contains morphological features of various bean species. The raw dataset contains 7 classes[1]; however, it is programmatically filtered down to exactly 3 distinct classes (Seker, Horoz, and Dermason) to provide a focused classification task. To evaluate the algorithm within a 2D spatial feature space, exactly two physical dimensions (Major Axis Length and Minor Axis Length) are extracted. These specific features naturally form overlapping spatial clusters, ensuring the decision boundaries are non-trivial.

3.2 The Scaling Imperative

K-NN is a distance-based algorithm heavily reliant on Euclidean geometry. Because the major axis of a bean naturally possesses a wider numerical variance than its minor axis, the data must be standardized using a `StandardScaler`. Without scaling, the dimension with the larger raw numbers would artificially dominate the distance calculations. The scaler is strictly fitted only to the training data to prevent information leakage.

3.3 Visualizing the Decision Boundary

By establishing a dense grid of coordinates across the 2D feature space using `numpy.meshgrid`, the algorithm is queried to predict the class of every coordinate. This technique allows for the direct visualization of the model's decision boundaries for specific values of $K \in \{1, 10, 25, 50, 100\}$.

3.4 The Hyperparameter Sweep

A loop executes the K-NN algorithm for every integer value of K from 1 to 100. The Training, Validation, and Testing errors are recorded and plotted directly against K to reveal the standard Bias-Variance tradeoff curves.

[1]The original Dry Bean dataset contains 7 classes (**Seker**, Barbunya, Bombay, Cali, **Dermason**, **Horoz**, Sira) and 16 continuous morphological features (Area, Perimeter, **Major Axis Length**, **Minor Axis Length**, Aspect Ratio, Eccentricity, Convex Area, Equiv Diameter, Extent, Solidity, Roundness, Compactness, ShapeFactor1, ShapeFactor2, ShapeFactor3, ShapeFactor4).

4. CODE IMPLEMENTATION

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier
from matplotlib.colors import ListedColormap
from sklearn.metrics import accuracy_score

# ==========================================
# PART 1: DATA LOADING & VISUALIZATION
# ==========================================

#Load Dry_Bean_Dataset.xlsx

df = pd.read_excel('Dry_Bean_Dataset.xlsx')

# OG dataset size
print(f"Original dataset size: {len(df)} rows")

# Clean the 'Class' column by removing hidden spaces and making it UPPERCASE
# This prevents 0-row filtering errors due to messy raw data strings

df['Class'] = df['Class'].astype(str).str.strip().str.upper()

# The original dataset has 7 classes. Filter it down to exactly 3 classes

df = df[df['Class'].isin(['SEKER', 'HOROZ', 'DERMASON'])]

print(f"Dataset size after filtering to 3 classes: {len(df)} rows")

# Map the 3 chosen species to integers (0, 1, 2)
class_map = {'SEKER': 0, 'HOROZ': 1, 'DERMASON': 2}
df['class_encoded'] = df['Class'].map(class_map)


# extract two physical dimensions for the 2D spatial map
# Feature 1: MajorAxisLength (Length of the bean)
# Feature 2: MinorAxisLength (Width of the bean)
X = df[['MajorAxisLength', 'MinorAxisLength']].values
y = df['class_encoded'].values

# Plotting the raw dataset
plt.figure(figsize=(10, 7))

# Scatter plot for each class
classes = ['Seker (Class 0)', 'Horoz (Class 1)', 'Dermason (Class 2)']
colors = ['#1f77b4', '#d62728', '#2ca02c'] # Blue, Red, Green

for i in range(3):
    plt.scatter(
        X[y == i, 0], X[y == i, 1],
        c=colors[i], label=classes[i],
        alpha=0.6, edgecolors='k', s=30
    )


plt.title('3-class Dry Bean Dataset: Physical Dimensions (2D Feature Space)',
    fontsize=14)
plt.xlabel('Major Axis Length', fontsize=12)
plt.ylabel('Minor Axis Length', fontsize=12)
plt.legend()
```

```python
plt.grid(True, alpha=0.3)
plt.show()


# ==========================================
# PART 2: PREPROCESSING & DATA SPLITTING
# ==========================================


# Splitting the filtered points into Train (70%), Validation (15%), and Testing (15%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
     # stratify = y -> ensures exact ratio of the classes is preserved
     # across all three splits.
)


X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=(0.15/0.85), random_state=42, stratify=y_temp
)


print(f"Train size: {len(X_train)} // Val size: {len(X_val)} // Test size: {len(X_test)}")

# Scaling prevents features with larger values from dominating the Euclidean distance.
scaler = StandardScaler()


# Fit the scaler only on the training data
X_train_scaled = scaler.fit_transform(X_train)

# Transform validation and test sets using the rules learned from the training set
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)


# ==========================================
# PART 3: VISUALIZING DECISION BOUNDARIES
# ==========================================

# Low values for K (K=1 or K=2) can be noisy and subject to the effects of outliers
# Large values for K smooth over things, but there is a possiblity that a category
# with only few samples in it will always be voted out by other categories

k_values = [1, 10, 25, 50, 100]


# Setup the color maps for the plot (Light colors for background, dark for points)
cmap_light = ListedColormap(['#cce5ff', '#ffcccc', '#d4f0d4']) # Light Blue, Light Red, Light Green
cmap_bold = ListedColormap(['#1f77b4', '#d62728', '#2ca02c'])  # Dark Blue, Dark Red, Dark Green


# 1. Create the Meshgrid
# We create a dense grid of points covering the entire feature space.
h = 0.05  # Step size of the grid
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))



plt.figure(figsize=(14, 18))
```

```python

# 2. Train and Plot for each K
for i, k in enumerate(k_values):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # Predict the class for every single point on our grid
    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # 3 rows, 2 columns grid arrangement for larger plots
    ax = plt.subplot(3, 2, i + 1)
    ax.contourf(xx, yy, Z, cmap=cmap_light)

    # Overlay the actual training points on top
    ax.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=y_train, cmap=cmap_bold,
               edgecolor='k', s=15, alpha=0.6)

    ax.set_title(f"Decision Boundary: K = {k}", fontsize=14)
    ax.set_xticks(())
    ax.set_yticks(())



plt.tight_layout()
plt.show()


# ==========================================
# PART 4: ERROR ANALYSIS
# ==========================================

# Testing every single K from 1 to 100
k_sweep = range(1, 101)
train_errors = []
val_errors = []
test_errors = []

for k in k_sweep:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # Calculate Error Rate (1 - Accuracy)
    train_err = 1 - accuracy_score(y_train, knn.predict(X_train_scaled))
    val_err = 1 - accuracy_score(y_val, knn.predict(X_val_scaled))
    test_err = 1 - accuracy_score(y_test, knn.predict(X_test_scaled))

    train_errors.append(train_err)
    val_errors.append(val_err)
    test_errors.append(test_err)

# Find the optimal K (the one with the lowest validation error)
best_k = k_sweep[np.argmin(val_errors)]


plt.figure(figsize=(12, 7))

plt.plot(k_sweep, train_errors, label='Training Error', color='blue', lw=2)
plt.plot(k_sweep, val_errors, label='Validation Error', color='green', lw=2)
plt.plot(k_sweep, test_errors, label='Testing Error', color='red', linestyle='--', lw
    =2)

# Highlight the 5 specific K values plotted boundaries for earlier
k_markers = [1, 10, 25, 50, 100]
for mk in k_markers:
    plt.axvline(x=mk, color='gray', linestyle=':', alpha=0.7)
```

```python
    plt.text(mk, max(val_errors), f'K={mk}', rotation=90, verticalalignment='top')

# Highlight the best K
plt.axvline(x=best_k, color='black', linestyle='-', label=f'Optimal K ({best_k})')


plt.title('Error Rate vs. K (Number of Neighbors)', fontsize=14)
plt.xlabel('K (Number of Neighbors)', fontsize=12)
plt.ylabel('Error Rate (1 - Accuracy)', fontsize=12)

plt.legend()
plt.grid(True, alpha=0.3)
print()
print(f"Optimal K value for the dataset {best_k}")
plt.show()
```

5. RESULTS AND OBSERVATIONS

5.1 Dataset Filtering and Splitting

```
Original dataset size: 13611 rows
Dataset size after filtering to 3 classes: 7501 rows
Train size: 5250 // Val size: 1125 // Test size: 1126
```

Console Output: Data Splitting

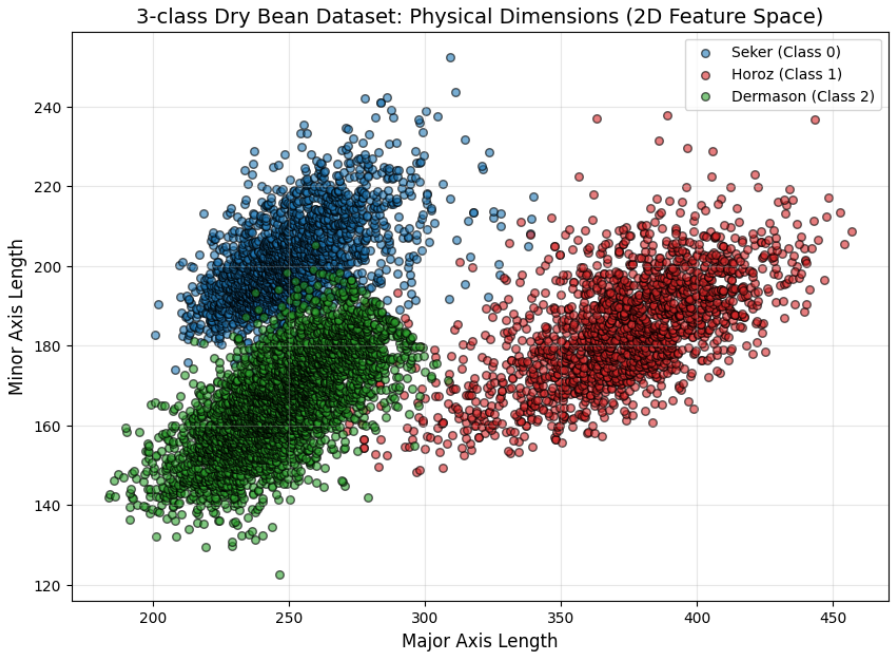

Figure 1: Real-World 3-class Dry Bean Dataset showing overlap between classes.

5.2 Decision Boundaries

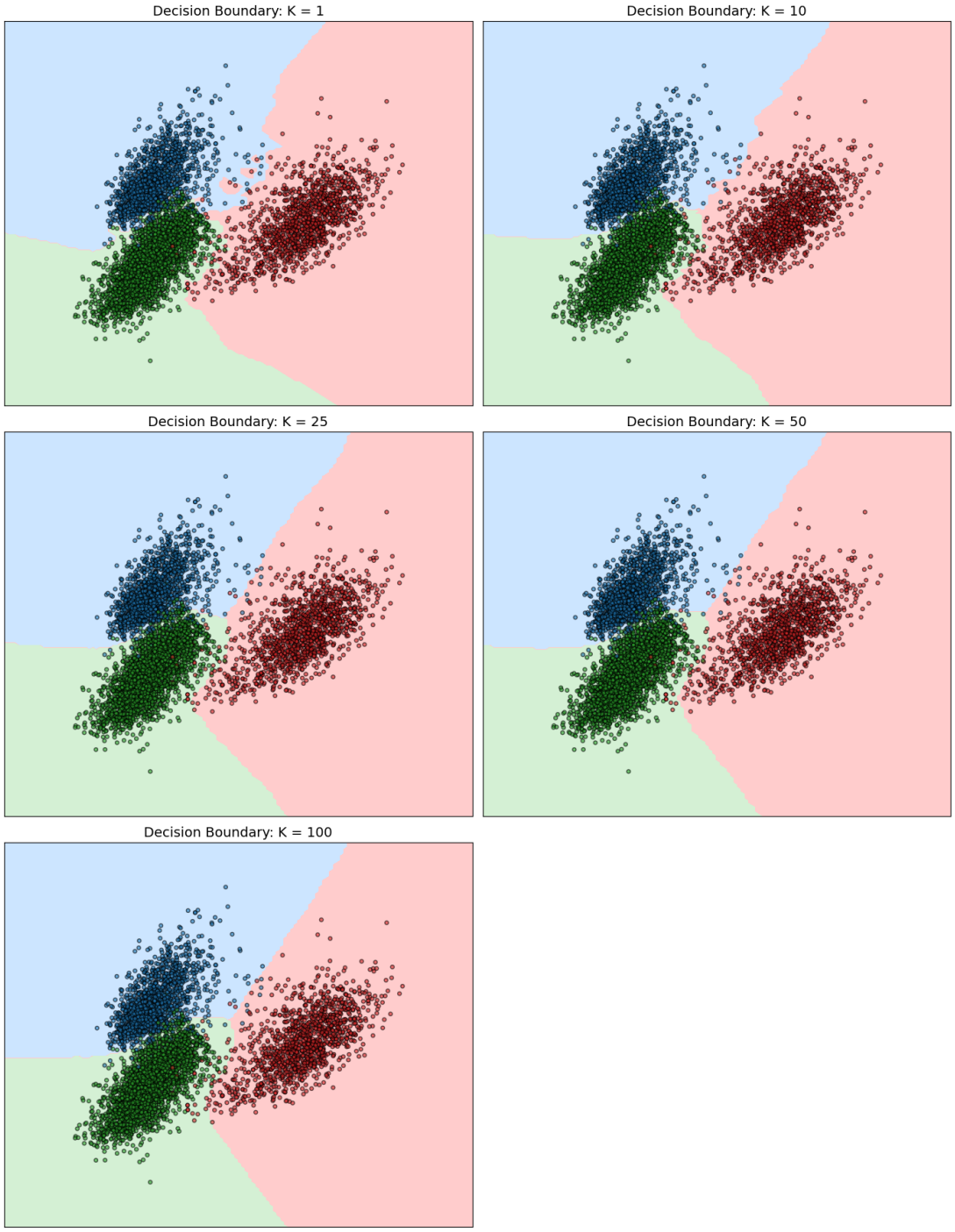

Figure 2: K-NN Decision Boundaries for K $\in \{1, 10, 25, 50, 100\}$.

5.3 The Bias-Variance Tradeoff

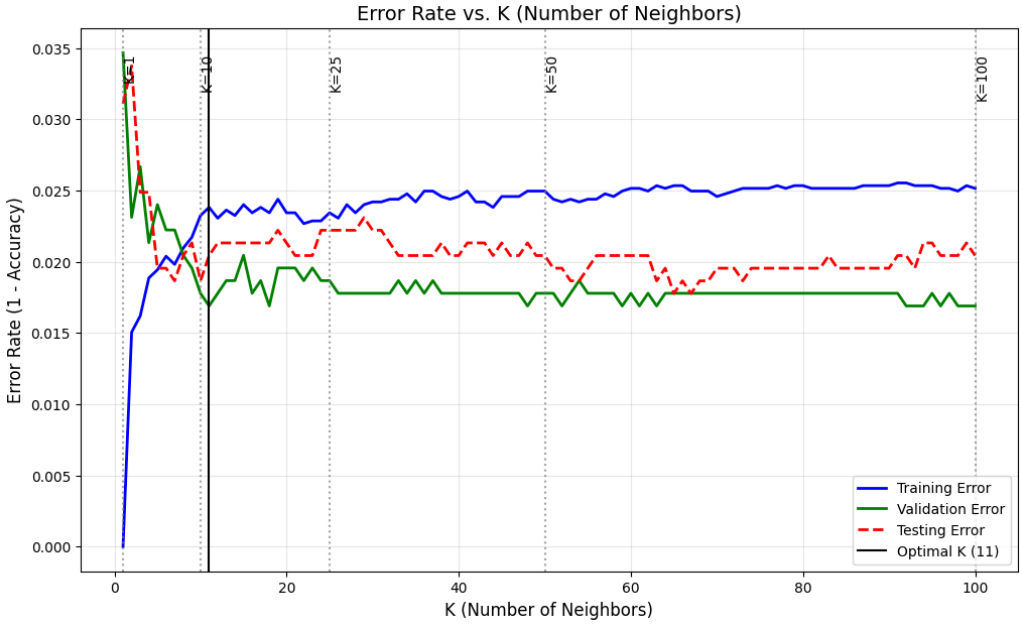

Figure 3: Error Rate vs. K (Number of Neighbors).

```
Optimal K value for the dataset: 11
```

Console Output: Optimal K